

len(zone)

-

Learning Bounded Latent Memory States for Long-Horizon Prediction

Enzo COGNÉVILLE*

Abstract

The challenge in long-horizon prediction is not to remember everything, but to maintain the right state. We introduce **len(zone)**, a framework for learning a bounded, persistent **latent memory state** that progressively transforms a growing stream of observations into a structured internal representation. Rather than retaining the full history or compressing it into a single recurrent vector, len(zone) represents memory as a fixed set of latent slots that persist and evolve over time. Incoming information is routed to the memory slots it is most relevant to. Existing slots can be updated, new slots can be created when necessary, and redundant or low-value information can be merged or removed to keep the memory bounded.

The objective is to learn a state M_t that preserves the information from the history H_t required for future prediction,

$$p(Y_{\text{future}} \mid H_t) \approx p_{\theta}(Y_{\text{future}} \mid M_t),$$

thereby replacing an ever-growing history with a compact state that is approximately sufficient for the future. This perspective connects learned memory to belief-state representations in partially observed systems and suggests a way to recover an approximately Markovian representation from history-dependent observations.

The same principle applies across several sequential modeling settings. In **world models**, len(zone) can provide a structured latent state for tracking persistent entities, relationships, latent regimes, and uncertainty under partial observability. In **large language models**, it can serve as a long-term memory layer that integrates information leaving the active context into a bounded state while preserving recent context explicitly. More broadly, len(zone) investigates whether selective updating, specialization, consolidation, and controlled forgetting can provide an effective mechanism for maintaining predictive state over long horizons.

*EDF R&D, cognenzo@gmail.com

1 Introduction

1.1 From growing histories to bounded states

The challenge in long-horizon prediction is not to remember everything, but to maintain the right state. In many sequential problems, the current observation is not sufficient to determine what should happen next.

An object may temporarily disappear from view while remaining relevant to future dynamics, a language model may need information introduced thousands of tokens earlier to answer a later query, and a noisy observation may be consistent with several possible hidden states whose identity can only be inferred by accumulating evidence over time. In all of these cases, prediction depends not only on the present observation, but also on information accumulated over the preceding history.

Let $H_t = (o_0, o_1, \dots, o_t)$ denote the observation history available at time t .

A direct solution is to preserve H_t itself. This retains the original information, but the amount of stored information grows continuously with the sequence length. Transformers make this trade-off explicit: extending the active context preserves access to more past tokens, but increases memory and computational requirements.

Rolling or sliding-window approaches bound these costs by retaining only a recent suffix, at the price of discarding information once it leaves the active window. Recurrent models instead summarize the past into a fixed-dimensional hidden state, providing bounded memory [Chou et al. \[2014\]](#), [Gu and Dao \[2024\]](#). However, all relevant information is compressed into the same state vector, whether it concerns different entities, variables, or events.

Modern sequence models explore a broader range of strategies, including explicit memory, context summarization, pruning, latent compression, and external retrieval. These approaches make different trade-offs between memory capacity, computational cost, exact retention, and the ability to revise previously stored information. A detailed review is provided in [Section 2](#).

The perspective taken in this work is that an expanding history should, when the structure of the problem permits it, be replaceable by a bounded state that preserves the information required for future prediction. We therefore seek a learned state M_t satisfying, approximately,

$$p(Y_{\text{future}} \mid H_t) \approx p_{\theta}(Y_{\text{future}} \mid M_t),$$

where Y_{future} denotes future quantities relevant to the task. The objective is not to reconstruct the entire past from memory, but to preserve the information from that past that remains useful for future prediction. Under a fixed memory budget, the state cannot in general retain every detail of an ever-growing history.

1.2 Predictive state and approximate Markovianity

A central idea in sequential modeling is that a process may depend on a long observation history while still admitting a compact state that summarizes the information from that history required for future prediction. A familiar example is the GARCH model used in financial time series. In a GARCH(1, 1) model, the conditional variance evolves according to

$$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2,$$

where r_t denotes the observed return. By recursively expanding this equation, the current variance can be interpreted as summarizing information from an increasingly long history of past squared returns, with increasingly distant observations receiving progressively smaller weights. Rather than explicitly retaining this

entire history, the model recursively updates the conditional variance σ_t^2 as new observations arrive [Bollerslev \[1986\]](#). The conditional variance therefore acts as a compact recursively updated representation of the past for future volatility prediction.

The same principle appears more generally in state-space and partially observed models. For a hidden Markov process with latent state S_t , the observation sequence itself need not be Markovian. However, the filtering belief

$$b_t(s) = P(S_t = s \mid H_t)$$

summarizes the information contained in the observation history that is required for predicting future observations under the assumed model [Rabiner \[1989\]](#). Similarly, in linear Gaussian state-space models, the Kalman filter recursively propagates a finite-dimensional posterior state rather than repeatedly processing the complete observation history [Kalman \[1960\]](#).

These examples illustrate an important distinction between the observed process and the state used to predict it. The observations may depend on a long history, while an appropriately chosen internal state can provide a recursive representation.:

$$H_t \longrightarrow M_t, \quad (M_t, o_{t+1}) \longrightarrow M_{t+1}.$$

The relevant question is therefore not necessarily how much of the history should be stored explicitly, but whether the information from that history that matters for future prediction can be represented by a compact state.

In classical models such as GARCH, hidden Markov models, or Kalman filters, the form of this state and its update rule are specified in advance. For example, the GARCH conditional variance σ_t^2 is chosen explicitly as the quantity that carries information about past volatility, and its recursion is imposed by the model. In more complex environments, however, it may not be known beforehand which variables should constitute the predictive state or how they should be updated.

This motivates the perspective adopted in this work. Rather than prescribing the relevant state, we seek to learn a latent memory state M_t directly from observations such that

$$p(Y_{\text{future}} \mid H_t) \approx p_{\theta}(Y_{\text{future}} \mid M_t).$$

The objective is therefore to learn a recursively maintainable representation that captures the information from the history that remains useful for future prediction.

In this sense, M_t is intended to summarize the past well enough that future prediction can rely primarily on M_t rather than on the full observation history. This does not require M_t to recover the true hidden state of the underlying system. Different internal representations may support the same predictions, and the information that must be preserved may depend on the task. The central requirement is predictive: once M_t has been constructed, access to the full history should provide as little additional useful information as possible.

1.3 **len(zone): a structured latent memory state**

We introduce **len(zone)**, a framework for maintaining a bounded, persistent, and structured latent memory state. Instead of representing memory by a single recurrent vector, **len(zone)** uses a fixed collection of K

latent slots,

$$M_t = \begin{bmatrix} m_t^1 \\ m_t^2 \\ \vdots \\ m_t^K \end{bmatrix} \in \mathbb{R}^{K \times d_m},$$

where d_m is the width of each slot. The size of the latent memory state is therefore bounded independently of the total length of the processed history.

The motivation for this structure is that long sequences may contain several persistent pieces of information that evolve at different timescales and should not necessarily be modified together. When a new observation arrives, a bounded memory system must therefore make several decisions. It must determine which existing parts of memory are relevant to the new information, whether that information should modify an existing representation or create a new one, how redundant representations should be handled, and what should eventually be discarded when memory capacity becomes limited.

These decisions motivate a simple memory lifecycle:

$$\text{ROUTE} \rightarrow \text{UPDATE} / \text{ALLOCATE} \rightarrow \text{CONSOLIDATE} \rightarrow \text{FORGET}.$$

ROUTE determines which memory slots are most relevant to the incoming information. If the information is compatible with an existing slot, UPDATE integrates it into that slot; otherwise, ALLOCATE can assign a new slot when capacity is available. As memory evolves, CONSOLIDATE can combine redundant or overlapping representations, while FORGET removes information whose expected predictive utility is sufficiently low. Together, these operations allow the memory state to evolve while remaining within a fixed capacity budget.

A slot is not assumed to correspond to a human-interpretable concept. Depending on the task, it may encode information about an entity, a relation, a latent regime, a goal, a constraint, or uncertainty over several alternatives. What matters is not whether the slot admits a convenient verbal label, but whether it forms a persistent and selectively useful unit of predictive state. This distinction is important because structured storage alone does not guarantee meaningful specialization. The routing mechanism, learning objective, capacity constraints, and interaction between slots must jointly create a pressure for useful organization.

The intended behavior can be illustrated by a simple example. Suppose a sequence first states that a red car belongs to Alice, later reveals that the car is electric, and eventually reports that Alice sold it to Bob. The first observation may lead to the creation of a memory representation associated with the car. When the fact that the car is electric appears later, the new information should be routed back to the same representation and incorporated without rewriting unrelated parts of memory. The later ownership change should similarly revise the relevant information while preserving previous ownership when it remains useful for future tasks. If multiple slots come to represent the same underlying entity, they may eventually be consolidated, while details that no longer contribute to future prediction may be forgotten.

The problem is therefore not merely one of retaining old observations. It involves deciding where new information belongs, what should be revised, when new memory should be created, and how a bounded state should reorganize itself as information accumulates.

1.4 A common principle across sequential models

The same memory principle is relevant to several classes of sequential models.

In *world models*, observations are generally partial views of an evolving environment. A useful state must preserve information about variables that may not be directly observable at the current time, such as occluded objects, persistent relationships, latent regimes, or uncertainty about alternative states. Latent-state world models already rely on recurrent state representations for prediction and planning [Hafner et al. \[2019\]](#). `len(zone)` investigates whether explicitly structured and selectively updated latent memory states can provide a useful alternative when several persistent factors must be maintained simultaneously. Connections to object-centric latent representations and slot-based dynamics provide additional motivation for this structure [Locatello et al. \[2020\]](#), [Wu et al. \[2023\]](#).

For *large language models*, the corresponding problem arises when useful information outlives the active context window. Rather than treating the entire past as permanent context, `len(zone)` can maintain a bounded long-term latent memory state alongside a recent verbatim context. When older content leaves the active window, its relevant information can be integrated into the persistent state, which is then reused and updated across subsequent context-overflow events. This differs from simply preserving an ever-growing cache and from repeatedly generating a textual summary: the persistent state itself is a learned object that can be selectively revised as new information arrives.

These settings differ in their observations, training objectives, and readout mechanisms, but they share the same underlying question: can an ever-growing history be replaced by a bounded, evolving state without losing the information that matters for future computation? `len(zone)` is intended as a general framework for studying this question rather than as a memory mechanism tied to a single modality or model family.

1.5 Contributions

This work makes the following contributions:

- We formulate long-horizon memory as the problem of learning a bounded latent memory state that preserves the task-relevant predictive information contained in an expanding history.
- We connect this objective to belief-state filtering and predictive sufficiency, motivating latent memory as an approximate Markovian state representation for history-dependent observations.
- We introduce **len(zone)**, a structured latent memory state composed of persistent slots whose contents can be selectively routed, updated, allocated, consolidated, and forgotten under a fixed capacity budget.
- We show how the same formulation naturally extends to partially observed world models and long-context language models, two settings in which maintaining useful state over long horizons is central.

The remainder of the paper reviews related approaches before formalizing the predictive-state objective, introducing the `len(zone)` architecture, and evaluating its behavior across long-horizon memory tasks.

2 Related Work

Long-horizon prediction has motivated a broad range of approaches for retaining, compressing, or recovering information from the past. Although these methods are often grouped under the general notion of “memory”, they operate on different representations and address different problems.

The categories reviewed below are therefore not mutually exclusive. Modern architectures often combine several of these mechanisms. The purpose of this section is instead to distinguish the main ways in which past information can be represented and made available to future computation before positioning len(zone) relative to them.

2.1 Recurrent and state-space models

A classical approach to long-range dependency is to replace the growing observation history with a recursively updated internal state. Recurrent neural networks maintain a representation of the form

$$h_t = U_\theta(h_{t-1}, x_t),$$

where x_t is the current input and h_t is a fixed-dimensional hidden state. Architectures such as LSTMs and GRUs introduce gating mechanisms that regulate how information is incorporated into or removed from this state [Hochreiter and Schmidhuber, 1997, Cho et al., 2014]. Once h_t has been computed, the complete preceding observation sequence does not need to be stored explicitly for the recurrence to continue.

State-space models rely on a closely related principle but impose a more structured dynamical form on the state transition. In a classical linear state-space model, the internal state follows a transition such as

$$h_t = Ah_{t-1} + Bx_t.$$

Modern neural state-space architectures adapt this principle to high-dimensional sequence modeling while preserving efficient recurrent computation. Selective state-space models such as Mamba make parts of the state transition dependent on the current input, allowing the model to regulate how incoming information affects the propagated state [Gu and Dao, 2024].

The same general principle appears in latent-state world models. There, a compact internal state is propagated through time and used to predict future observations or support planning under partial observability [Hafner et al., 2019]. The relevant state may therefore summarize information that is no longer directly observable in the current input.

Predictive State Representations provide a complementary perspective. Rather than defining state through an underlying physical latent variable, they characterize it through quantities sufficient for predicting future observations [Littman et al., 2001]. This emphasizes that the usefulness of an internal state can be defined through its predictive role rather than through a direct correspondence with the true hidden variables of the environment.

The common principle across these approaches is that an increasingly long observation history can be replaced by a finite-dimensional state that is propagated as new information arrives.

2.2 Memory-augmented architectures

A second family of approaches supplements the main computational model with an explicit memory that can be read from and written to over time.

Neural Turing Machines introduce a differentiable memory matrix together with learned mechanisms for addressing, reading, and writing its contents [Graves et al., 2014]. A controller interacts with multiple memory locations through continuous attention weights, allowing information to be stored outside its own recurrent hidden state.

Differentiable Neural Computers extend this framework with additional mechanisms for memory allocation, usage tracking, and temporal links between memory locations [Graves et al., 2016]. The memory can therefore support not only content-based retrieval but also structured access to previously stored information.

Related ideas have subsequently been incorporated into Transformer architectures. Memformer maintains a fixed number of persistent memory slots that are carried across successive input segments [Wu et al., 2022]. Recurrent Memory Transformer similarly introduces dedicated memory tokens that persist from one segment to the next, allowing information to survive beyond a local attention window [Bulatov et al., 2022].

Other architectures combine persistent memory with compression. Compressive Transformer retains a recent memory while transforming older activations into a lower-resolution representation [Rae et al., 2020]. This allows information to remain accessible after leaving the short-term memory without retaining all past activations at full resolution.

More recent approaches explore memories that adapt during inference. Titans introduces a neural long-term memory updated through test-time learning, allowing information encountered during a sequence to modify a persistent memory mechanism [Behrouz et al., 2025].

These architectures demonstrate that sequential models can maintain explicit, persistent, and addressable information outside a single recurrent hidden state. Unlike KV-cache techniques discussed later, this principle is not inherently restricted to language models and can be applied more generally to sequential or partially observed systems.

2.3 Structured and slot-based latent representations

A complementary line of work concerns the internal organization of latent state. Instead of representing an observation or environment through a single monolithic vector, object-centric approaches decompose it into multiple latent components.

Slot Attention maps a set of input features to a fixed number of latent slots through iterative attention and recurrent refinement [Locatello et al., 2020]. The slots compete for information from the input and can specialize to different objects or coherent parts of a visual scene without requiring explicit object-level supervision.

SlotFormer extends this principle to temporal dynamics [Wu et al., 2023]. Rather than predicting future observations directly in pixel space, it models the evolution and interaction of object-centric slots over time. The state of a scene can therefore be represented as a collection of latent components whose dynamics jointly determine future predictions.

These methods are particularly relevant to world models because they demonstrate that a predictive latent state need not be monolithic. Different parts of the state can represent different persistent components of the environment and evolve through time. More generally, slot-based representations provide evidence that structured latent units can emerge from learning objectives without requiring each slot to be assigned a predefined semantic interpretation.

2.4 External and retrieval-based memory

Another strategy is to preserve past information outside the model’s current internal state and retrieve it when needed.

In its general form, this approach maintains an external archive of observations, experiences, documents, or previously constructed memories. A query derived from the current state is then used to select a subset of this archive for the present computation. This paradigm is widely used in language modeling. Retrieval-augmented systems combine language models with external corpora or datastores, allowing information to be recovered without keeping the complete corpus in the active context. kNN-LM and RETRO, for example, augment language modeling with information retrieved from large external stores [Khandelwal et al., 2020, Borgeaud et al., 2022].

External memory can also be constructed dynamically from previous interactions. MemGPT organizes information across different memory tiers and moves content between the active context and longer-term storage [Packer et al., 2023].

The same principle is not restricted to language. An agent or world model can similarly maintain an episodic archive of previous observations or trajectories and retrieve relevant experiences when encountering a related situation.

Retrieval therefore makes a different trade-off from bounded state representations. Instead of deciding continuously how all relevant information should be encoded inside a compact state, detailed information can remain stored externally and selection can be postponed until a later query reveals what is needed.

2.5 Context reduction in long-context language models

A separate family of methods is more specific to Transformer-based language models. Rather than constructing a general predictive state, these approaches reduce the amount of explicit context or cached activation history that must remain available during inference.

Context reduction can take several forms:

summarization, pruning, compression.

Summarization and token-level reduction. The most direct approach is to replace an old portion of the context with a shorter natural-language summary. Because the resulting representation consists of ordinary tokens, it can be inserted directly into the context of an existing language model.

Other approaches operate at the token level and selectively remove parts of the context considered less useful. Such *pruning* reduces the amount of information retained explicitly without constructing a new latent representation. The central challenge is to determine which elements can be safely removed, since information that appears unimportant at one time may become relevant later.

Latent context compression. Rather than returning to natural language, latent compression methods map long contexts into a smaller set of continuous learned representations.

AutoCompressors recursively transform preceding segments into learned summary vectors that are reused while processing later context [Chevalier et al., 2023]. In-Context Autoencoder (ICAE) similarly encodes an input sequence into a small number of latent memory slots that can subsequently be consumed by a language model [Ge et al., 2024]. Activation Beacon introduces dedicated beacon tokens whose internal activations represent compressed portions of a long context at different compression ratios [Zhang et al., 2025].

These approaches replace many explicit tokens with a substantially smaller latent representation while remaining compatible with Transformer computation.

KV-cache pruning and compression. During autoregressive generation, each Transformer layer stores key and value representations of previously processed tokens in a KV cache. This avoids recomputing the full prefix at every decoding step, but the cache grows with the retained context length.

One way to control this growth is through pruning. H₂O preserves recent tokens together with tokens that receive large accumulated attention, referred to as “heavy hitters”, while evicting lower-priority cache entries [Zhang et al., 2023]. StreamingLLM retains a recent window together with a small number of initial tokens that act as attention sinks, enabling stable processing of long token streams under a bounded cache [Xiao et al., 2024].

Other methods perform compression rather than hard pruning. LoCoCo maintains a bounded KV cache by adaptively combining previous and incoming key–value representations [Cai et al., 2024]. Dynamic Memory Compression (DMC) similarly learns to compress the cache online, with compression behavior varying across layers and attention heads [Nawrot et al., 2024].

These techniques are primarily motivated by the computational structure of Transformer inference. Their representations are tied to tokens, attention layers, or cached activations, and therefore do not directly apply to sequential models that do not use Transformer-style attention. Their broader principles (selective retention, pruning, and compression) are nevertheless relevant to the general problem of bounded memory.

3 Predictive State Formulation

The previous sections motivate replacing an expanding observation history with a bounded internal state. We now formalize the property that such a state should satisfy independently of the particular memory architecture used to construct it.

The central object is a latent memory state M_t derived from the information available up to time t . Its purpose is not to reconstruct the complete past, but to preserve the information from that past that remains relevant for future prediction.

3.1 Sequential setting and notation

Let

$$H_t = (o_0, o_1, \dots, o_t)$$

denote the history available at time t in an observation-only setting.

In controlled sequential systems, such as world models, the history may additionally contain actions,

$$H_t = (o_0, a_0, o_1, a_1, \dots, a_{t-1}, o_t),$$

where action a_t is chosen after observing o_t and affects subsequent observations.

A memory mechanism maps the available history to a latent state,

$$M_t = \Phi_\phi(H_t),$$

where Φ_ϕ denotes the memory construction process, parameterized by ϕ .

The objective is to replace the growing history H_t with a bounded state M_t that preserves the information required for future computation.

3.2 Task-relative predictive sufficiency

To define what information the memory should preserve, let Y denote a future quantity that the model may need to predict. Depending on the application, Y may correspond to a future observation, a hidden-state estimate, an answer to a later query, a reward, or another task-dependent target.

We additionally introduce a variable Q describing the context in which this prediction is requested. For example, Q may specify a forecast horizon, a future action sequence, or a downstream query.

An ideal memory state would preserve all information from H_t that is relevant for predicting Y once the prediction context Q is specified. Formally, exact predictive sufficiency would require

$$p(Y \mid H_t, Q) = p(Y \mid M_t, Q),$$

This condition does not require the complete history to be recoverable from M_t . Two histories may therefore produce the same memory state whenever the differences between them have no effect on the future predictions of interest.

In practice, exact sufficiency is generally unattainable. We therefore seek an approximately sufficient state satisfying

$$p(Y \mid H_t, Q) \approx p_\psi(Y \mid M_t, Q),$$

where $p_\psi(Y \mid M_t, Q)$ denotes the predictive distribution produced from the memory state M_t and prediction context Q , with ψ denoting the parameters of the predictive readout.

When the full predictive distribution is available, the approximation can be measured using the Kullback–Leibler divergence,

$$\mathcal{D}_{\text{pred}} = \mathbb{E}_{H_t, Q} [D_{\text{KL}}(p(Y | H_t, Q) \parallel p_\psi(Y | M_t, Q))].$$

Here, $D_{\text{KL}}(p \parallel q)$ denotes the Kullback–Leibler divergence between the reference predictive distribution p and the approximation q .

A useful memory should make this quantity small under the distribution of histories and future questions relevant to the task.

The dependence on Q is important. A representation that is sufficient for predicting the next observation need not preserve the information required to answer an exact question thousands of steps later. Similarly, a state optimized for one control objective may discard information required by another. Predictive sufficiency is therefore defined relative to the future predictions the memory is expected to support.

When Q is not known at the time the memory is written, the writer cannot condition its state update on the future query itself. Instead, training must encourage the state to preserve information that is useful across the relevant distribution of possible future queries.

3.3 Belief states as an exact reference

Partially observed Markov systems provide a canonical example in which such a predictive state is known analytically.

Let S_t denote a latent Markov state and let s denote a possible value of S_t . The filtering belief is then

$$b_t(s) = p(S_t = s | H_t)$$

be the filtering belief after observing history H_t . Under the assumed model, this posterior contains the information from the observation history required for future prediction.

For example, if Q specifies a future action sequence or prediction condition, then

$$p(Y | H_t, Q) = \int p(Y | S_t = s, Q) b_t(s) ds.$$

The complete observation history is therefore unnecessary once the filtering belief is known. In a finite-state Hidden Markov Model, this belief is a probability vector [Rabiner, 1989]. In a linear Gaussian state-space model, the corresponding posterior can be represented recursively through its mean and covariance using the Kalman filter [Kalman, 1960].

These examples provide an exact reference for the predictive-state perspective. They do not, however, imply that a learned memory state must explicitly represent a probability distribution or recover the true hidden state.

A learned representation M_t may encode the same predictive information in a different coordinate system. Several distinct latent representations may therefore be equally useful if they induce the same future predictions. This distinction is particularly important in complex environments, where neither the relevant hidden variables nor the appropriate posterior representation are known in advance. The objective is then to learn a state that plays a role analogous to a belief state without prescribing its semantics beforehand.

3.4 Recursive state and approximate Markovianity

For long-running systems, it is not sufficient for M_t to be compact. It should also be maintainable without repeatedly processing the complete history.

We therefore require a recursive update of the form

$$M_{t+1} = U_\phi(M_t, e_{t+1}),$$

where e_{t+1} denotes the new information incorporated at the next memory update.

In an environment model, e_{t+1} may correspond to a new observation together with the preceding action. In a language-model application, a memory update may instead process an entire block of tokens leaving the active context window. The notion of an update event is therefore application-dependent, while the recursive principle remains the same.

Combining recursive maintenance with predictive sufficiency gives

$$H_t \longrightarrow M_t \longrightarrow p_\theta(Y \mid M_t, Q),$$

with future state updates depending on the previous memory rather than on the complete preceding history. We refer to this as an *approximately Markovian representation*. The term is used here in a predictive sense: after conditioning on M_t and Q , access to the older history should provide little additional information about the future quantities of interest.

This does not imply that the observed process itself is Markovian, nor that M_t recovers the true physical state of the system. The requirement is weaker and operational: M_t should contain enough information from the past for the future computations that the model is expected to perform.

3.5 Bounded predictive compression

Predictive sufficiency alone does not define a memory problem. The trivial representation

$$M_t = H_t$$

preserves all available information but grows indefinitely with the sequence length.

The setting considered in this work therefore imposes a fixed deployment budget. At finite precision, we require

$$\text{bytes}(M_t) \leq B_{\text{mem}} \quad \text{for all } t,$$

where B_{mem} is independent of the total processed history length.

The resulting problem can be written abstractly as

$$\min_{\phi, \psi} \mathbb{E} [\ell(Y, D_\psi(M_t, Q))] \quad \text{subject to} \quad \text{bytes}(M_t) \leq B_{\text{mem}},$$

where D_ψ denotes a task-dependent readout and ℓ the corresponding predictive loss.

When a reference predictive distribution is available, the same objective can instead be expressed through $\mathcal{D}_{\text{pred}}$. In applications where no exact reference exists, the predictive objective may be instantiated through future-observation likelihood, supervised task loss, behavioral distillation, or other task-specific criteria.

4 len(zone) Architecture

We now introduce the architecture used by len(zone) to maintain such a state over time.

The central design choice is to represent memory as a collection of persistent latent units rather than as a single recurrent vector. New information does not necessarily rewrite the complete state. Instead, the memory is maintained through a sequence of selective operations:

$$\text{ROUTE} \rightarrow \text{UPDATE} / \text{ALLOCATE} \rightarrow \text{CONSOLIDATE} \rightarrow \text{FORGET}.$$

These operations should be understood as a state-maintenance framework rather than as a requirement that every operation be executed after every observation. For example, consolidation and forgetting may be triggered only when memory becomes sufficiently redundant or capacity is under pressure.

4.1 Design principles

The len(zone) architecture is built around four properties:

- **Boundedness.** The number and dimensionality of memory slots remain fixed as the processed history grows.
- **Persistence.** Information stored in a slot can survive across many subsequent observations without being rewritten.
- **Selective modification.** New evidence can modify a subset of the memory rather than the complete state.
- **Predictive maintenance.** Allocation, consolidation, and forgetting are ultimately evaluated according to their effect on future predictive utility.

4.2 Memory state and readout

The latent memory consists of K slots,

$$M_t = \begin{bmatrix} m_t^1 \\ m_t^2 \\ \vdots \\ m_t^K \end{bmatrix} \in \mathbb{R}^{K \times d_m},$$

where $m_t^k \in \mathbb{R}^{d_m}$ denotes slot k .

An occupancy vector

$$c_t = (c_t^1, \dots, c_t^K) \in \{0, 1\}^K$$

indicates which slots currently contain valid memory. Inactive slots are excluded from memory reads and routing decisions. For notational simplicity, we continue to refer to M_t as the latent memory state, with c_t understood as bounded auxiliary state required for its management.

The memory writer is parameterized by ϕ and recursively transforms the current state and new evidence,

$$(M_{t+1}, c_{t+1}) = U_\phi(M_t, c_t, e_{t+1}).$$

The predictive reader is kept conceptually separate. Given memory M_t and prediction context Q , it produces

$$p_\psi(Y \mid M_t, Q).$$

This separation is useful because the mechanism that maintains memory need not be identical to the mechanism that consumes it. A world model may decode future observations from the state, while a language model may consume projected memory representations together with its recent context.

Slots are not assumed to correspond to individual objects or human-readable concepts. A slot may represent information about an entity, relation, latent regime, event, constraint, goal, or uncertainty, and several slots may jointly encode the same predictive factor. What matters is whether the organization of the memory supports persistent and selective use of information.

4.3 ROUTE: selecting relevant memory

When new information arrives, the first problem is to determine whether it relates to information already represented in memory.

The new information is first encoded as

$$x_t = E_\phi(e_t).$$

For each memory slot, the router computes a compatibility score

$$s_t^k = R_\phi(x_t, m_{t-1}^k), \quad k = 1, \dots, K.$$

A simple implementation can use the standard scaled dot-product attention mechanism,

$$s_t^k = \frac{q_\phi(x_t)^\top k_\phi(m_{t-1}^k)}{\sqrt{d_k}},$$

where q_ϕ and k_ϕ project the incoming information and memory slots into a shared comparison space. Only occupied slots should participate in routing. We therefore define the masked compatibility scores

$$\tilde{s}_t^k = \begin{cases} s_t^k, & \text{if } c_{t-1}^k = 1, \\ -\infty, & \text{if } c_{t-1}^k = 0. \end{cases}$$

A softmax over the existing memory slots alone would always assign the highest probability to one of them, even when none provides a good match for the incoming information. Moreover, not every incoming piece of information should necessarily be stored in persistent memory.

We therefore introduce two additional learned routing alternatives. The first is a NEW key, k_ϕ^{new} , representing the creation of a new memory unit:

$$s_t^{\text{new}} = \frac{q_\phi(x_t)^\top k_\phi^{\text{new}}}{\sqrt{d_k}}.$$

The second is a NOWRITE key, k_ϕ^{nowrite} , representing the decision not to modify persistent memory:

$$s_t^{\text{nowrite}} = \frac{q_\phi(x_t)^\top k_\phi^{\text{nowrite}}}{\sqrt{d_k}}.$$

The occupied slots and the two routing alternatives then compete in a single routing distribution,

$$\pi_t = \text{softmax} \left(\frac{1}{\tau} [\tilde{s}_t^1, \dots, \tilde{s}_t^K, s_t^{\text{new}}, s_t^{\text{nowrite}}] \right),$$

where τ controls the concentration of the routing distribution.

The routing decision has three possible outcomes. Selecting an existing slot triggers UPDATE. Selecting NEW requests the creation of a new memory unit, while selecting NOWRITE leaves the persistent memory unchanged.

If NEW is selected and

$$\sum_{k=1}^K c_{t-1}^k < K,$$

a free slot is available and the system proceeds directly to ALLOCATE. If all slots are occupied, selecting NEW instead creates a capacity conflict that must be resolved through consolidation, forgetting, or replacement before allocation.

If an existing memory slot is selected, we condition the routing distribution on the existing slots. The resulting routing weights are

$$\alpha_t^k = \frac{\pi_t^k}{1 - \pi_t^{\text{new}} - \pi_t^{\text{nowrite}}}, \quad k = 1, \dots, K,$$

with $\alpha_t^k = 0$ for unoccupied slots.

The NEW alternative indicates that the incoming information is better represented by a new memory unit than by updating an existing slot. If

$$\sum_{k=1}^K c_{t-1}^k < K,$$

a free slot is available and selecting NEW directly triggers ALLOCATE.

If all K slots are occupied, selecting NEW instead signals a capacity conflict, which must first be resolved through consolidation, forgetting, or replacement before a new slot can be allocated. Otherwise, the probabilities associated with the existing slots can be renormalized to obtain routing weights

$$\alpha_t^k = \frac{\pi_t^k}{1 - \pi_t^{\text{new}}}, \quad k = 1, \dots, K,$$

with $\alpha_t^k = 0$ for unoccupied slots.

Routing may be dense, but a selective implementation can restrict expensive updates to a small subset of the most relevant slots,

$$\mathcal{I}_t = \text{Top}_r(\alpha_t), \quad r \ll K.$$

The routing mechanism therefore determines both whether the incoming information should create a new memory unit and, when it should not, which existing slots are most relevant for updating. More expressive implementations may additionally condition the novelty score on the compatibility scores of the existing slots.

4.4 UPDATE: revising existing memory

If the routing decision is UPDATE, only the selected memory slots $k \in \mathcal{I}_t$ are considered for modification, while the remaining slots are left unchanged.

For each selected slot k , the update mechanism receives its previous content m_{t-1}^k together with the encoded incoming information x_t . Rather than directly reconstructing the complete slot representation, the update function proposes a latent modification

$$\Delta_t^k = F_\phi(m_{t-1}^k, x_t),$$

where $\Delta_t^k \in \mathbb{R}^{d_m}$ represents the change suggested by the incoming information.

The model also computes an update gate

$$g_t^k = \sigma \left(G_\phi(m_{t-1}^k, x_t) \right),$$

where $\sigma(u) = 1/(1 + e^{-u})$ denotes the sigmoid function. It maps the output of G_ϕ to the interval $(0, 1)$, so that g_t^k controls how strongly the proposed modification should affect the existing memory.

Combining the update gate with the routing weight, we define

$$\lambda_t^k = \alpha_t^k g_t^k.$$

The selected slot is then updated through a residual modification,

$$m_t^{k,+} = m_{t-1}^k + \lambda_t^k \Delta_t^k, \quad k \in \mathcal{I}_t.$$

Slots that are not selected are carried forward unchanged:

$$m_t^{k,+} = m_{t-1}^k, \quad k \notin \mathcal{I}_t.$$

The superscript $+$ denotes the intermediate memory state obtained after incorporating the current information, but before any subsequent CONSOLIDATE or FORGET operation.

Selective updating alone, however, does not guarantee that different slots will learn distinct or stable roles. The learning objectives used to encourage useful memory organization are introduced in Section 5.

4.5 ALLOCATE: creating new memory

If the router determines that the incoming information is not adequately represented by an existing slot, the system may allocate a new memory unit.

When an inactive slot k is available,

$$m_t^{k,+} = A_\phi(x_t), \quad c_t^k = 1,$$

where A_ϕ initializes the new latent representation from the incoming evidence.

Because the total number of slots is fixed,

$$\sum_{k=1}^K c_t^k \leq K,$$

allocation eventually creates capacity pressure.

When no free slot remains, selecting NEW creates a capacity conflict. The system must then recover capacity before allocation, for example through CONSOLIDATE or FORGET.

4.6 Capacity management

When NEW is selected while all K memory slots are occupied, the system must recover capacity before the incoming information can be stored. `len(zone)` considers two main capacity-recovery operations: `CONSOLIDATE`, which combines information from multiple slots, and `FORGET`, which removes a slot whose expected future utility is sufficiently low.

Let $\bar{M}_t$ denote the full memory state at the beginning of capacity management.

CONSOLIDATE. A consolidation operation replaces two existing slots by a single latent representation. For candidate slots i and j , a learned merge operator constructs

$$m_t^\star = C_\phi \left(\bar{m}_t^i, \bar{m}_t^j \right).$$

Replacing the two source slots by $m_t^\star$ frees one memory location.

Considering every possible pair may require up to

$$\binom{K}{2} = \mathcal{O}(K^2)$$

candidate evaluations. We therefore separate inexpensive candidate proposal from the subsequent evaluation of the merge.

A simple proposal mechanism projects the slots into a consolidation space and computes a compatibility score

$$\rho_{ij} = \frac{h_\phi(\bar{m}_t^i)^\top h_\phi(\bar{m}_t^j)}{\sqrt{d_c}},$$

where h_ϕ is a learned projection and d_c is the dimension of the consolidation space.

Only a small number of high-scoring pairs are retained:

$$\mathcal{C}_t = \text{Top}_B(\{\rho_{ij} : i < j\}), \quad B \ll K^2.$$

For large memories, the proposal stage may instead rely on approximate nearest-neighbor search, avoiding an exhaustive comparison of all pairs.

The score ρ_{ij} is used only to identify plausible consolidation candidates. Geometric similarity alone does not imply that two slots can be safely merged, since nearby latent representations may still carry different predictive information.

FORGET. Forgetting directly releases an occupied slot k . Unlike consolidation, this produces only K possible candidate operations, so candidate generation is linear in the number of memory slots.

The usefulness of a slot cannot in general be determined from its content alone. Its contribution depends on the rest of the memory: a slot may be dispensable when equivalent information is represented elsewhere, but important otherwise. Forgetting decisions should therefore be evaluated in the context of the current memory state.

Unified capacity-recovery decision. The consolidation and forgetting candidates are collected into a common action set

$$\mathcal{A}_t = \{\text{MERGE}(i, j) : (i, j) \in \mathcal{C}_t\} \cup \left\{ \text{DROP}(k) : c_t^k = 1 \right\}.$$

Each action $a \in \mathcal{A}_t$ frees one memory slot. After applying the action, the incoming representation x_t can be allocated to the released location. Let

$$T_a(\bar{M}_t, x_t)$$

denote the resulting memory state after applying capacity-recovery action a and storing x_t .

The quality of a capacity-management decision can then be characterized by its effect on future predictive performance. Conceptually, we define

$$\Delta_t(a) = \mathbb{E}_{Y,Q} [\ell(Y, D_\psi(T_a(\bar{M}_t, x_t), Q)) - \ell(Y, D_\psi(\bar{M}_t, Q))],$$

where ℓ is the task-dependent predictive loss.

The quantity $\Delta_t(a)$ measures the net predictive effect of freeing capacity using action a and incorporating the new information. Smaller values therefore correspond to preferable capacity-management decisions.

This formulation is independent of a particular predictive domain. For a language model, ℓ may correspond to token-prediction cross-entropy; for a world model, it may measure future-state or rollout prediction error. When probabilistic predictive distributions are available, the same comparison may instead be expressed using a Kullback–Leibler divergence.

The exact quantity $\Delta_t(a)$ generally cannot be computed during inference because future targets and queries are unknown. `len(zone)` therefore uses a learned capacity-management scorer

$$\hat{\Delta}_t(a) = S_\phi^{\text{cap}}(\bar{M}_t, x_t, a)$$

to estimate the predictive cost associated with each candidate action.

The same scorer is used for both consolidation and forgetting candidates, so that their estimated costs can be compared directly. The selected capacity-recovery operation is

$$a_t^* = \arg \min_{a \in \mathcal{A}_t} \hat{\Delta}_t(a).$$

The selected operation is applied, freeing one slot, after which x_t is allocated to the resulting memory state. Optionally, a `NOWRITE` fallback can be included in $\mathcal{A}_t$. This allows the system to preserve the current memory and abandon the allocation of x_t when every available consolidation or forgetting operation is predicted to cause excessive loss.

5 Learning and Calibration

5.1 Predictive objective

The architecture introduced in Section 4 specifies how a bounded latent memory state can be maintained, but does not by itself determine which information should be stored, updated, or discarded.

The central learning signal remains predictive. The memory writer should organize the latent state so that the predictive reader can perform future tasks using the bounded memory rather than the complete preceding history.

We therefore use the task-dependent predictive objective

$$\mathcal{L}_{\text{pred}} = \mathbb{E}_{H_t, Y, Q} [\ell(Y, D_\psi(M_t, Q))],$$

where

$$M_t = \Phi_\phi(H_t),$$

ϕ denotes the parameters of the memory writer, ψ the parameters of the predictive reader, and ℓ the predictive loss associated with the application.

This objective provides the ultimate criterion for all memory decisions: routing, updating, allocation, consolidation, and forgetting are useful only insofar as they improve future predictive performance under the fixed memory budget.

5.2 Learning the routing policy

The routing mechanism ultimately makes discrete decisions, but applying hard routing from the beginning of training would prevent gradients from flowing through alternative memory decisions.

During training, we therefore retain the soft routing distribution

$$\pi_t = (\pi_t^1, \dots, \pi_t^K, \pi_t^{\text{new}}, \pi_t^{\text{nowrite}})$$

and use it to provide a differentiable learning signal to the router.

As training progresses, the routing distribution can be made increasingly selective by decreasing its temperature τ or by replacing soft routing with a straight-through discrete estimator. At inference time, the learned policy can use hard routing decisions and sparse Top- r slot selection.

5.3 Calibrating capacity management

Capacity-management decisions require estimating the predictive cost of alternative consolidation and forgetting operations. During deployment, this cost must be predicted from the current memory, but during training future observations or targets are available and can be used to construct counterfactual supervision.

For a sampled candidate action a , we apply the corresponding capacity-management operation and evaluate its effect over a future training horizon. This provides a target

$$\Delta_t(a) = \mathcal{L}_{\text{future}}(T_a(\bar{M}_t, x_t)) - \mathcal{L}_{\text{future}}(\bar{M}_t),$$

where $\mathcal{L}_{\text{future}}$ denotes the predictive loss evaluated on future targets.

The capacity-management scorer is then trained to predict this quantity, for example through

$$\mathcal{L}_{\text{cap}} = \mathbb{E}_{t,a} \left[\left(\hat{\Delta}_t(a) - \Delta_t(a) \right)^2 \right].$$

The scorer therefore learns, from observed future consequences during training, to estimate which memory modification is least harmful when the future is no longer available at inference time.

5.4 Memory-organization regularization

The predictive objective does not necessarily guarantee sparse or stable memory behavior. Optional auxiliary regularizers may therefore be introduced when required by the optimization dynamics.

A routing sparsity term can encourage decisions to concentrate on a small number of slots, while an update regularizer can discourage unnecessary changes to persistent memory. Such terms should remain secondary to the predictive objective: slot specialization is not imposed geometrically, but should emerge when it improves predictive performance under the bounded capacity constraint.

5.5 Training procedure

The resulting training objective can be written as

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \beta_{\text{cap}} \mathcal{L}_{\text{cap}} + \beta_{\text{aux}} \mathcal{L}_{\text{aux}},$$

where $\mathcal{L}_{\text{aux}}$ collects optional regularization terms such as routing sparsity or update stability.

The predictive objective remains the primary learning signal. The additional terms are used only to make the discrete memory-management mechanisms easier to learn and deploy efficiently.

6 Extensions and Research Directions

The experiments considered in this work focus on the simplest integration of len(zone) into a language model: the latent memory state is represented as a set of soft tokens prepended to the recent context,

$$[M_t \mid X_{\text{recent}}].$$

This design has the advantage of requiring only limited modifications to a standard Transformer architecture and provides a direct way to test whether a bounded structured latent memory state can preserve useful information beyond the active context window.

Several natural extensions follow from this basic formulation.

6.1 Multi-timescale memory

The current formulation treats all memory slots symmetrically with respect to their update dynamics. However, useful information may evolve at very different timescales. Some aspects of the state may remain stable over long horizons, while others may need to be revised frequently.

This suggests partitioning the memory into groups with different characteristic update rates,

$$M_t = [M_t^{\text{slow}}, M_t^{\text{medium}}, M_t^{\text{fast}}].$$

A simple implementation could impose fixed update schedules, with fast memory updated every chunk and slower memory updated less frequently. A more general formulation could instead learn slot-specific update rates or writing costs, allowing the hierarchy of timescales to emerge from data.

Such a mechanism would test whether explicitly separating stable and volatile information improves long-horizon retention, reduces unnecessary rewriting, or provides a more efficient memory organization. A natural experimental comparison would therefore contrast a single-timescale memory, a hand-designed multi-timescale hierarchy, and a learned hierarchy of update frequencies.

6.2 Direct KV-memory integration

Representing memory as soft tokens makes it part of the causal token sequence. When the memory state changes, previously computed representations of the recent context may therefore become inconsistent with the updated prefix, requiring a new prefill.

An alternative is to expose the latent memory state to the Transformer through a separate memory-attention mechanism. At layer ℓ , the memory slots could be projected directly into memory-specific keys and values,

$$K_{\text{mem}}^\ell = M_t W_{K,\text{mem}}^\ell, \quad V_{\text{mem}}^\ell = M_t W_{V,\text{mem}}^\ell,$$

which are consulted alongside the standard KV cache associated with the recent context.

This formulation would decouple long-term memory from the causal token prefix. If only a subset of memory slots changes, only the corresponding memory keys and values would need to be recomputed, while the recent-context KV cache could remain unchanged. The resulting architecture raises a separate set of questions, including whether memory should be exposed at every Transformer layer or only at selected depths, and how the additional memory attention affects inference cost.

6.3 Sparse and routed memory access

Direct attention to all memory slots becomes increasingly expensive as the memory capacity grows. A scalable extension is therefore to distinguish between the full latent memory state and a smaller active subset used for the current computation.

Let

$$M_t = \{m_t^1, \dots, m_t^K\}$$

denote the complete memory. A routing mechanism could select

$$M_t^{\text{active}} \subset M_t, \quad |M_t^{\text{active}}| \ll K,$$

based on the current query, token block, or local context. Only these active slots would then participate in memory attention.

This leads to a hierarchical view of long-term state:

$$M_t^{\text{long}} \longrightarrow M_t^{\text{active}} \longrightarrow X_{\text{recent}}.$$

The long-term memory contains the complete bounded predictive state, the active memory contains the subset currently relevant to computation, and the recent context preserves exact short-term information.

Such a design could allow the total memory capacity to increase without requiring every token to attend to every stored slot. It also gives the routing mechanism a dual role: routing would determine not only where new information should be written, but also which parts of the memory should be read for the current computation.

6.4 A broader research program

These extensions address three distinct questions about bounded latent memory:

Multi-timescale memory: *At what timescale should different parts of memory evolve?*

KV-memory integration: *How should a model access persistent memory efficiently?*

Sparse memory access: *How can the memory capacity scale without proportional attention cost?*

They are intentionally left outside the main experimental scope of the present work. The objective here is first to establish the behavior of the simpler soft-token formulation and to evaluate whether maintaining a bounded structured latent memory state is useful in itself. The extensions above provide natural directions for studying the temporal organization, computational integration, and scalability of such memory in future work.

References

- Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. In *Advances in Neural Information Processing Systems*, volume 38, pages 125925–125962. Neural Information Processing Systems Foundation, Inc., 2025. doi: 10.52202/085713-3786.
- Tim Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3): 307–327, 1986. doi: 10.1016/0304-4076(86)90063-1.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240. PMLR, 2022.
- Aydar Bulatov, Yury Kuratov, and Mikhail Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems*, volume 35, pages 11079–11091. Curran Associates, Inc., 2022. doi: 10.52202/068431-0805.
- Ruisi Cai, Yuandong Tian, Zhangyang Wang, and Beidi Chen. Lococo: Dropping in convolutions for long context compression. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 5348–5359. PMLR, 2024.
- Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.232.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734. Association for Computational Linguistics, 2014. doi: 10.3115/v1/D14-1179.
- Tao Ge, Hu Jing, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations (ICLR)*, 2024.
- Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016. doi: 10.1038/nature20101.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling (COLM)*, 2024.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2555–2565. PMLR, 2019.

- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960. doi: 10.1115/1.3662552.
- Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.
- Michael L. Littman, Richard S. Sutton, and Satinder Singh. Predictive representations of state. In *Advances in Neural Information Processing Systems*, volume 14. MIT Press, 2001.
- Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems*, volume 33, pages 11525–11538. Curran Associates, Inc., 2020.
- Piotr Nawrot, Adrian Łańcucki, Marcin Chochowski, David Tarjan, and Edoardo M. Ponti. Dynamic memory compression: Retrofitting llms for accelerated inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 37396–37412. PMLR, 2024.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Lawrence R. Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.
- Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.
- Qingyang Wu, Zhenzhong Lan, Kun Qian, Jing Gu, Alborz Geramifard, and Zhou Yu. Memformer: A memory-augmented transformer for sequence modeling. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2022*, pages 308–318. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.findings-acl.29.
- Ziyi Wu, Nikita Dvornik, Klaus Greff, Thomas Kipf, and Animesh Garg. Slotformer: Unsupervised visual dynamics simulation with object-centric models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024.
- Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. Long context compression with activation beacon. In *International Conference on Learning Representations*, 2025.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H₂o: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

7 Annexes

7.1 Erreurs / typographies observées

1. Introduction
2. Related Work and Positioning
3. Predictive State Formulation
4. len(zone) Architecture
5. Learning Objectives
6. Applications / Experiments